The Islamia University of Bahawalpur

Hotel Management System

A project presented to

Department of Information Technology, IUB Bahawalpur

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Information Technology 2022-2026

By

Sharjeel Abid

Roll No: F22BINFT1M01219

2022-2026

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Sharjeel Abid

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (IT) "**Hotel Management System**" was developed by

Sharjeel Abid , F22BINFT1M01219, 2022-2026 under the supervision of

"**Mam Kainat**" and that in (his/her) opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Information Technology.

Supervisor

External Examiner

Chairman Department of Information Technology

Acknowledgement

Give acknowledgment to all who helped to complete this project.

Sharjeel Abid

Abbreviations

Provide a list of all abbreviation used in the document.

Abbreviation	Full Form
SRS	Software Requirements Specification
SDD	Software Design Document
UI	User Interface
UX	User Experience

Table of Contents

Chapter 1
Introduction

This Software Requirements Specification (SRS) presents a clear overview of the Hotel Booking Management System and defines what the system must do from both functional and non-functional perspectives.

Key points of this introduction are:

- The document focuses only on requirements and does not include implementation or design details.
- It identifies the system's purpose, scope, users, constraints, assumptions, and interfaces.
- It provides a shared understanding for supervisors, developers, testers, and stakeholders.
- It ensures that all required system behaviors are clearly defined before development and testing begin.
- It serves as a baseline for validation, traceability, and planning future enhancements.

1.1 Purpose

This Software Requirements Specification (SRS) document defines the requirements for the **Hotel Booking Management System**, a web-based application designed to support hotel room searching, reservation, payment processing, and booking management.

The main purpose of this document is to clearly specify what the system is expected to do, so that development, testing, and maintenance activities can be performed in an organized and systematic manner.

Intended Audience:

- Project supervisor and examiners
- Developers and system maintainers
- Quality assurance (QA) testers
- Hotel management stakeholders and administrators

1.2 Scope

The software product is titled **Hotel Booking Management System**. It provides a digital platform for both hotel customers and administrators.

The system will:

- Enable users to register, log in, search for rooms, view room details, and make bookings
- Support secure online payments through an integrated payment gateway
- Allow users to view booking history and invoices
- Allow administrators to manage rooms, bookings, users, and system settings

The system will not:

- Handle flight or ticket booking services
- Support multi-hotel chain integration in the current version
- Provide multilingual support in the current version
- Include advanced analytics or AI-based recommendations

The primary objective is to reduce manual booking effort, increase booking efficiency, and ensure transparency for both customers and hotel staff.

1.3 Definitions, Acronyms, and Abbreviations

- **SRS:** Software Requirements Specification
- **SDD:** Software Design Document
- **UI/UX:** User Interface / User Experience
- **API:** Application Programming Interface
- **DBMS:** Database Management System
- **CRUD:** Create, Read, Update, Delete
- **HTTP/HTTPS:** Web communication protocols
- **AJAX:** Asynchronous JavaScript and XML (used for asynchronous requests)
- **Session:** Server-side user state maintained after login
- **Booking Order:** Primary reservation record stored in the database
- **Payment Callback:** Gateway response endpoint used to verify payment status

1.4 References

- Project SRS document (submitted version)
- Project SDD document (submitted version)
- Stripe API Documentation
- MySQL Documentation
- PHP Documentation
- Bootstrap Documentation
- React and Node.js references from initial requirement context (for comparison and planning)

1.5 Overview

This SRS document is structured as follows:

- **Chapter 1:** Introduction including scope, context, users, constraints, and assumptions

- **Chapter 2:** Detailed software requirements (functional and non-functional)
- **Chapter 3:** Mapping of requirements to system design and architecture
- **Chapter 4:** Implementation details, database integration, UI screens, and testing

Business stakeholders should focus on requirement sections, while developers and testers should concentrate on design, implementation, and validation sections.

1.6 Product Perspective

The system is a standalone web application designed for hotel booking operations. It replaces traditional manual booking methods such as phone calls, walk-ins, or spreadsheet-based handling with a fully integrated online system.

The system follows a modular web architecture consisting of:

- Customer-facing interface
- Admin control panel
- Shared backend services
- MySQL database
- Payment gateway integration

1.6.1 System Interfaces

External system interfaces include:

- Payment gateway interface (Stripe)
- Web browser-based user interface
- Database interface (MySQL)

1.6.2 User Interfaces

The system provides:

- Responsive web pages for customers (home, room listings, room details, booking, invoice, and history pages)
- Admin dashboard interfaces for managing rooms, bookings, and users
- Modal-based login/registration forms and real-time action feedback alerts

1.6.3 Hardware Interfaces

The system does not require any direct hardware interaction.
It operates on standard devices such as desktops, laptops, and mobile devices through a web browser.

1.6.4 Software Interfaces

- Server-side language: PHP
- Database: MySQL
- Web server: Apache (using XAMPP environment during development)
- Frontend technologies: Bootstrap, JavaScript, Swiper
- Payment integration: Stripe SDK/API

1.6.5 Communications Interfaces

- Communication between browser and server via HTTP/HTTPS
- Asynchronous client-server communication using AJAX (GET/POST)
- Payment processing communication through Stripe redirect and callback mechanism

1.6.6 Memory Constraints

No specific memory limitations are defined for this version.
The system assumes standard hosting resources suitable for small to medium-scale usage.

1.6.7 Operations

The system supports the following operational modes:

- Interactive user operations (searching, booking, payment)
- Interactive admin operations (CRUD management)
- Continuous system availability (business hours or 24/7 hosting)
- Routine backup and recovery through database exports and server maintenance

1.6.8 Site Adaptation Requirements

To deploy the system on a new environment:

- Configure web server and PHP runtime
- Set up or import the MySQL database schema
- Configure base application URL
- Add payment gateway credentials
- Verify email and payment redirection settings

1.7 Product Functions

Key system functionalities include:

- User registration and authentication
- Room listing, filtering, and availability checking
- Detailed room display including facilities and pricing
- Date-based booking process
- Payment processing and confirmation
- Booking history tracking and invoice generation
- Admin control for managing rooms, bookings, users, and system settings

1.8 User Characteristics

Primary user groups include:

- **Customer/User:**
  Has basic knowledge of web and mobile usage; performs searching, booking, and payments
- **Administrator:**
  Possesses moderate technical knowledge; responsible for managing rooms, bookings, and users

The system is designed for users with general digital literacy and does not require specialized technical skills.

1.9 Constraints

- The system must operate as a browser-based web application
- Requires stable internet connectivity for booking and payment operations
- Depends on payment gateway availability for transaction processing
- Data accuracy depends on correct configuration by the administrator
- Limited to a single system (no external integrations in current version)
- Security mechanisms and session management must be properly maintained

1.10 Assumptions and Dependencies

Assumptions:

- Users provide accurate registration and booking details
- Administrators maintain correct room and pricing data
- Hosting environment supports required PHP and MySQL features

Dependencies:

- Availability of MySQL database
- Availability of Stripe API and valid credentials
- Web server (Apache/XAMPP or equivalent)
- Accurate server time and date configuration

1.11 Apportioning of Requirements

Implemented in Current Version:

- User authentication
- Room search and detailed view
- Booking process
- Payment gateway integration
- Booking history and invoice generation
- Admin CRUD operations

Planned for Future Versions:

- Advanced reporting and analytics
- Multi-language support
- Multi-hotel or chain system integration
- Enhanced notifications (SMS/email improvements)
- Recommendation and personalization features

Chapter 2
Software Requirements Specification

2.1 Specific Requirements

The following requirements are uniquely identified, measurable, and expressed using “shall” statements.

2.1.1 Functional Requirements (FR)

FR-1 User Registration
The system shall allow a new user to register by providing name, email, phone number, address, pincode, date of birth, profile image, password, and confirm password.

FR-2 User Login/Logout
The system shall authenticate users using email/phone and password, establish a valid session upon successful login, and terminate the session on logout.

FR-3 Room Search and Filtering
The system shall enable users to view available rooms and apply filters based on check-in/check-out dates, number of adults, children, and available facilities.

FR-4 Room Detail View
The system shall display detailed information for each room, including images, features, facilities, area, guest capacity, and price per night.

FR-5 Booking Validation
The system shall validate booking inputs such that check-in is not in the past and check-out is later than check-in before proceeding.

FR-6 Booking and Payment Initialization
The system shall generate a booking order and corresponding booking details before redirecting the user to the payment gateway.

FR-7 Payment Processing
The system shall process payments through Stripe and update booking status only after successful payment verification via callback.

FR-8 Invoice and Booking History
The system shall allow logged-in users to view invoices and their booking history.

FR-9 Admin Room Management
The system shall allow administrators to add, update, activate/deactivate, and delete rooms, including managing associated images.

FR-10 Admin Booking and User Management
The system shall allow administrators to view and manage bookings and users, including performing necessary operational actions.

2.1.2 Priority Classification

- **High Priority:** FR-1, FR-2, FR-3, FR-5, FR-6, FR-7, FR-9
- **Medium Priority:** FR-4, FR-8, FR-10
- **Low/Future Priority:** Advanced analytics, multilingual support, recommendation engine

2.2 External Interfaces

2.2.1 User Interface Inputs/Outputs

- **Inputs:** Login credentials, registration data, filtering parameters, booking dates, payment actions
- **Outputs:** Room listings, availability results, booking confirmations, payment status, invoices, system alerts/messages

2.2.2 Payment Gateway Interface

- **Interface:** Stripe Checkout Session API
- **Input:** Order ID, payment amount, booking metadata
- **Output:** Success/cancel redirects and payment intent status

2.2.3 Database Interface

- **DBMS:** MySQL
- **Interface Method:** Server-side prepared statements and SQL queries
- **Output:** Stored user, room, booking, and payment-related records

2.2.4 Admin Interface

- **Input:** CRUD forms for rooms, users, and bookings
- **Output:** Updated data records, status changes, and confirmation messages

2.3 Functions

2.3.1 Authentication Functions

- Register user
- Login user
- Session validation
- Logout

2.3.2 Room Discovery Functions

- Display active room listings
- Filter rooms by guests, facilities, and dates
- Show detailed room information

2.3.3 Booking Functions

- Validate booking request data
- Calculate total cost based on number of nights × room price
- Create booking order and details
- Prevent invalid booking transitions

2.3.4 Payment Functions

- Create Stripe checkout session
- Handle payment callback
- Update payment and booking status
- Redirect to invoice/history

2.3.5 Admin Functions

- Room CRUD operations with image handling
- User listing and status management
- Booking monitoring and control actions
- System settings and content management

2.4 Performance Requirements

- **PR-1:** 95% of room listing and filtering requests shall respond within 3 seconds under normal load
- **PR-2:** The system shall support at least 100 concurrent users for browsing and searching
- **PR-3:** Booking-to-payment redirection shall complete within 5 seconds (excluding external gateway delays)

- **PR-4:** Invoice and booking history pages shall load within 3 seconds for standard data volume

2.5 Logical Database Requirements

2.5.1 Core Entities

- user_cred
- rooms
- room_images
- features
- facilities
- room_features
- room_facilities
- booking_order
- booking_details
- refund_requests (if enabled)

2.5.2 Relationship Requirements

- One user can have multiple bookings
- One room can have multiple bookings
- Rooms and facilities/features follow a many-to-many relationship via mapping tables
- Each booking order shall be linked with a booking details record

2.5.3 Integrity Constraints

- Unique email and phone for users
- Valid foreign key references for rooms in bookings
- Date consistency (check-out must be after check-in)
- Booking status updated only after verified payment

2.5.4 Data Retention

- Booking and payment records shall be retained for auditing and reporting
- Soft delete mechanisms shall preserve historical data integrity where required

2.6 Design Constraints

- The system shall function as a browser-based web application
- Backend shall support session-based authentication
- Payments shall be processed via secure gateway (Stripe)
- Database shall be compatible with MySQL
- Deployment requires proper server configuration and payment credentials

2.6.1 Standards Compliance

- Compliance with HTTP/HTTPS protocols
- Secure password storage using hashing (bcrypt)
- SQL queries implemented with input sanitization and prepared statements
- Booking and transaction status tracking for audit purposes
- Payment card handling managed via Stripe-hosted checkout (reducing PCI scope)

2.7 Software System Attributes

2.7.1 Reliability

- The system shall ensure consistent booking status transitions
- Failed payments shall not mark bookings as completed

2.7.2 Availability

- The system should remain available within defined uptime (target 24/7)
- Users should be able to safely retry booking or payment in case of failure

2.7.3 Security

- Protected sessions for authenticated access
- Secure password storage and validation
- Input validation and sanitization
- Clear separation between user and admin roles

2.7.4 Maintainability

- Modular project structure (inc, ajax, admin, shared components)
- Reusable database/helper functions
- Separation of frontend and admin workflows

2.7.5 Portability

- Compatible with standard PHP and MySQL hosting environments
- Accessible via web browsers on desktop and mobile
- No dependency on specialized hardware

2.8 Organizing the Specific Requirements

This SRS follows a **Feature + User Class** organization:

- **User Classes:** Customer, Administrator
- **Feature Groups:** Authentication, Room Discovery, Booking, Payment, Booking Records, Admin Management

This structure enhances clarity and traceability with system modules and test cases.

2.8.1 System Mode

- User mode
- Admin mode

2.8.2 User Class

- Customer
- Administrator

2.8.3 Objects

User, Room, Booking, Payment, Invoice, Facility, Feature

2.8.4 Feature

Register/Login, Search/Filter, Book/Pay, Manage Rooms/Bookings

2.8.5 Stimulus

Form submission, filtering actions, booking requests, payment callbacks

2.8.6 Response

Validation results, updated room listings, redirects, status updates, invoice generation

2.8.7 Functional Hierarchy

Authentication → Search → Room Details → Booking → Payment → History/Invoice
Admin Login → Dashboard → Room/User/Booking Management

2.9 Additional Comments

The current version focuses on the core booking lifecycle and administrative operations.

Advanced features such as multi-language support, AI-based recommendations, and enterprise integrations are planned for future versions.

All requirements are structured to be testable and traceable with corresponding implementation modules and test cases.

Chapter 3
System Design and Architecture

3.1 Architectural Design

The Hotel Booking Management System is designed as a modular web application with a clear separation of responsibilities.

At a high level, the system is divided into the following subsystems:

Presentation Layer (User Interface)

- User pages: Home, Rooms, Room Details, Booking History, Invoice
- Admin pages: Dashboard, Rooms Management, Bookings Management, Users Management
- Built using HTML, CSS, Bootstrap, and JavaScript

Application Logic Layer

- User-side handlers located in `ajax/`
- Admin-side handlers located in `admin/ajax/`
- Shared helpers and validation located in `admin/inc/` and `inc/bootstrap.php`
- Responsible for authentication, booking logic, room filtering, and payment flow

Data Layer

- MySQL database stores users, rooms, metadata, bookings, and payment data
- Accessed via reusable query functions in shared database modules

External Service Layer

- Stripe payment gateway for secure checkout and payment verification

Subsystem Collaboration (Working Flow)

- User submits action from UI (login, search, booking)
- Request is sent to backend handler (`ajax/...` or PHP logic)
- Backend validates data and queries MySQL
- Response is returned to UI (HTML update, redirect, alert, or status)
- For payment, backend communicates with Stripe and processes callback before final confirmation

Conceptual Diagram (Textual)
User/Admin Browser → UI Pages → PHP Handlers → MySQL Database
PHP Handlers ↔ Stripe API (for payment processing)

3.2 Decomposition Description

This system follows a functional decomposition approach.

3.2.1 Top-level Functional Modules

- Authentication Module
- Room Discovery Module
- Booking Module
- Payment Module
- Booking Records Module
- Admin Management Module
- Shared Utility Module

3.2.2 Functional Decomposition (DFD-style)

Input: User/Admin actions from forms and buttons

Processes:

- P1: Authenticate user/admin
- P2: Filter and display rooms
- P3: Validate booking and calculate amount
- P4: Handle payment initiation and verification
- P5: Store and retrieve booking records
- P6: Admin CRUD operations

Data Stores:

- D1 Users
- D2 Rooms
- D3 Room mappings/images
- D4 Booking orders/details
- D5 Settings/auxiliary tables

Output:

- Alerts (success/failure)
- Room listings and details
- Booking and invoice records
- Updated admin views

- **3.2.3 Main Sequence (Customer Booking Flow)**

- User logs in
- System validates and creates session
- User searches/selects room
- System checks availability and date conditions
- User confirms booking
- System creates booking records and Stripe session
- Stripe callback is received
- System updates payment status
- Invoice/history is shown

3.3 Design Rationale

The chosen architecture is based on simplicity, maintainability, and project timeline.

Reasons for Selection:

- Modular folder structure (`inc`, `ajax`, `admin`) ensures clear separation
- Shared helpers reduce code duplication
- Session-based authentication is simple and effective
- Stripe integration reduces payment complexity

Alternatives Considered:

- Monolithic single-file system → rejected (hard to maintain)
- Microservices architecture → rejected (too complex)
- SPA + API architecture → not chosen due to time constraints

Trade-offs:

- Easier and faster development
- Moderate scalability (can be improved later)

3.4 Data Description

The system data is structured into relational database entities and mapping tables.

Major Entities:

- User: authentication and profile data
- Room: room details, pricing, and status
- Features/Facilities: many-to-many mapping
- Room Images: gallery and thumbnails
- Booking Order: transaction state

- Booking Details: booking snapshot
- Settings/Contact: system configuration

Data Flow:

- Input is validated and processed via backend
- Session stores temporary booking data
- Persistent data is stored in MySQL

3.5 Data Dictionary (Core)

The following table provides a structured overview of the core database entities, their primary fields, data types, and functional roles within the system.

Entity / Table	Key Fields	Type	Description
user_cred	id, name, email, phonenum, password, status	Mixed	Customer account data including authentication and account status.
rooms	id, name, area, price, quantity, adult, children, status, removed	Mixed	Room master records containing capacity, pricing, and availability attributes.
room_images	sr_no, room_id, image, thumb	Mixed	Storage for room-related image paths and thumbnail identifiers.
features	id, name	Mixed	Catalog of specific room characteristics (e.g., Sea View, Balcony).
facilities	id, name, icon	Mixed	Catalog of hotel-wide amenities and their respective icons (e.g.,

Entity / Table	Key Fields	Type	Description
			WiFi, Pool).
room_features	room_id, features_id	Int-Int	Mapping table linking rooms to their specific features.
room_facilities	room_id, facilities_id	Int-Int	Mapping table linking rooms to available hotel facilities.
booking_order	booking_id, user_id, room_id, check_in, check_out, order_id, booking_status, trans_status	Mixed	Primary transaction record tracking the lifecycle of a booking and payment.
booking_details	booking_id, room_name, price, total_pay, user_name, phonenum, address	Mixed	Snapshot of booking information at the time of purchase (used for reporting/invoices).
settings	sr_no, site_title, site_about, shutdown	Mixed	Global configuration settings, including site-wide descriptions and maintenance mode.
contact_details	sr_no, fb, insta, tw, etc.	Mixed	Storage for official contact information and social media integration links.

3.6 Component Design

3.6.1 Authentication Component

Files: `inc/header.php`, `ajax/login_register.php`, `logout.php`

Login Logic:

- Receive credentials
- Query user
- Validate status
- Verify password
- Set session

Register Logic:

- Validate input
- Check uniqueness
- Upload image
- Hash password
- Insert record

3.6.2 Room Discovery Component

Files: `rooms.php`, `ajax/filter_rooms.php`

Logic:

- Read filters
- Build query
- Exclude booked rooms
- Fetch and display results

3.6.3 Booking Component

Files: `room_details.php`, `confirm_booking.php`

Logic:

- Validate inputs
- Calculate amount
- Store session data
- Redirect to payment

3.6.4 Payment Component

Files: `stripe_payment.php`, `pgRedirect.php`

Logic:

- Verify session
- Create booking records
- Start Stripe session
- Handle callback
- Update status

3.6.5 Booking Records Component

Files: `bookings.php`, `invoice.php`

Logic:

- Fetch user bookings
- Determine status
- Display history and invoice

3.6.6 Admin Component

Files: `admin/*.php`, `admin/ajax/*.php`

Logic:

- Validate admin session
- Perform CRUD operations
- Return updated views

Chapter 4
Implementation and Testing

4.1 Code Modules

Common/Bootstrap

- `inc/bootstrap.php`
- `admin/inc/db_config.php`
- `admin/inc/essentials.php`

User Interface

- `index.php`, `rooms.php`, `room_details.php`, `bookings.php`, `invoice.php`

Backend/AJAX

- `ajax/login_register.php`
- `ajax/filter_rooms.php`
- `ajax/confirm_booking.php`

Payment

- Stripe integration files

Admin

- Admin panel files and handlers

4.2 Database Connectivity

- Connection initialized in shared config
- Uses reusable query functions
- Data sanitized before queries
- Booking/payment stored in database

4.3 Overview of User Interface

User Flow:

- User opens home page
- Registers/logs in
- Searches rooms
- Views details

- Books and pays
- Receives invoice and history

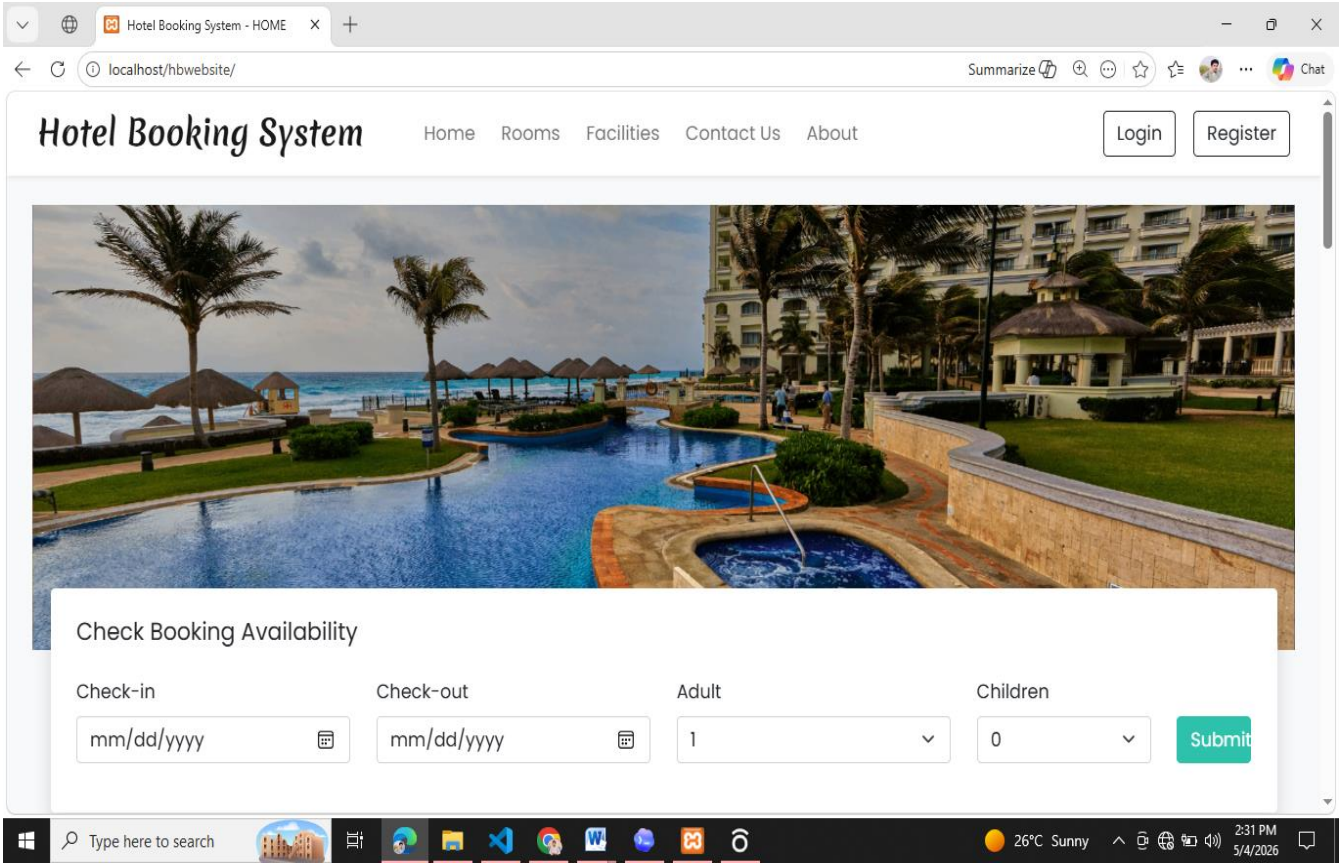
Admin Flow:

- Admin logs in
- Manages rooms, users, bookings
- Updates settings

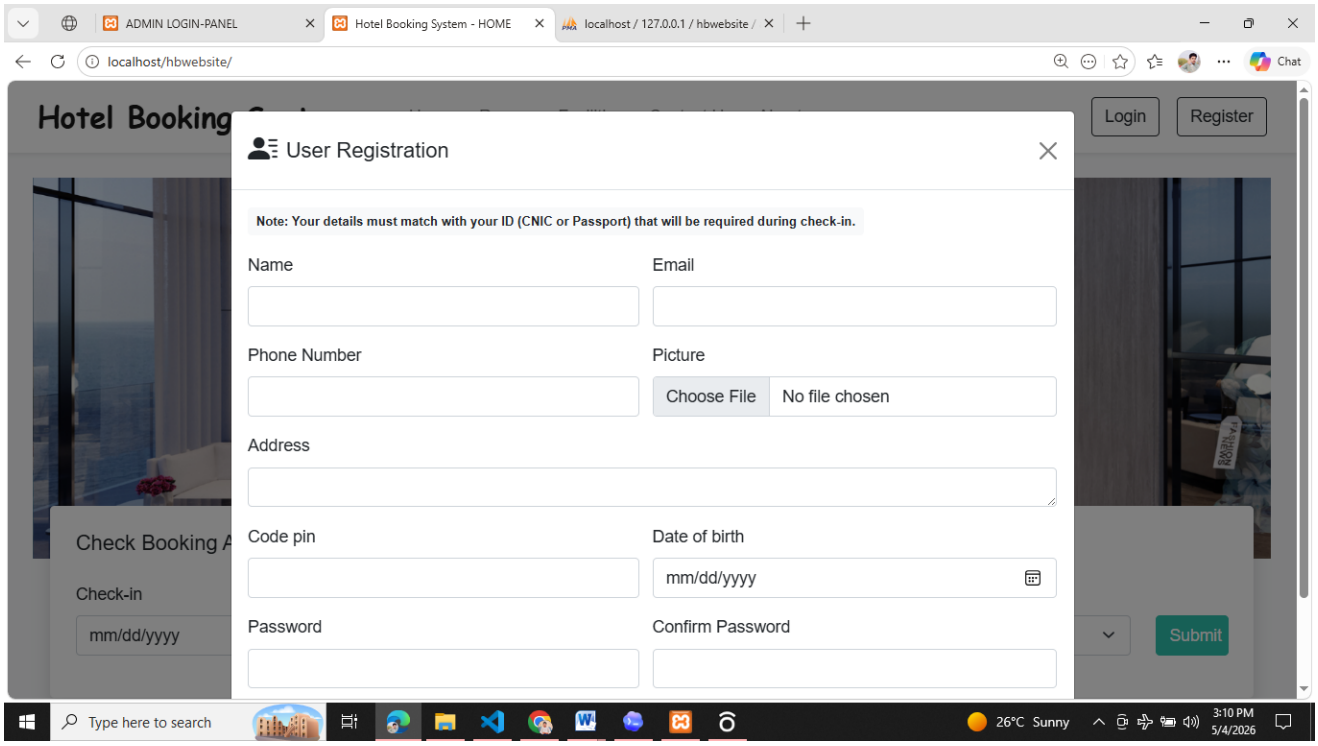
4.4 Screen Images

Include screenshots for:

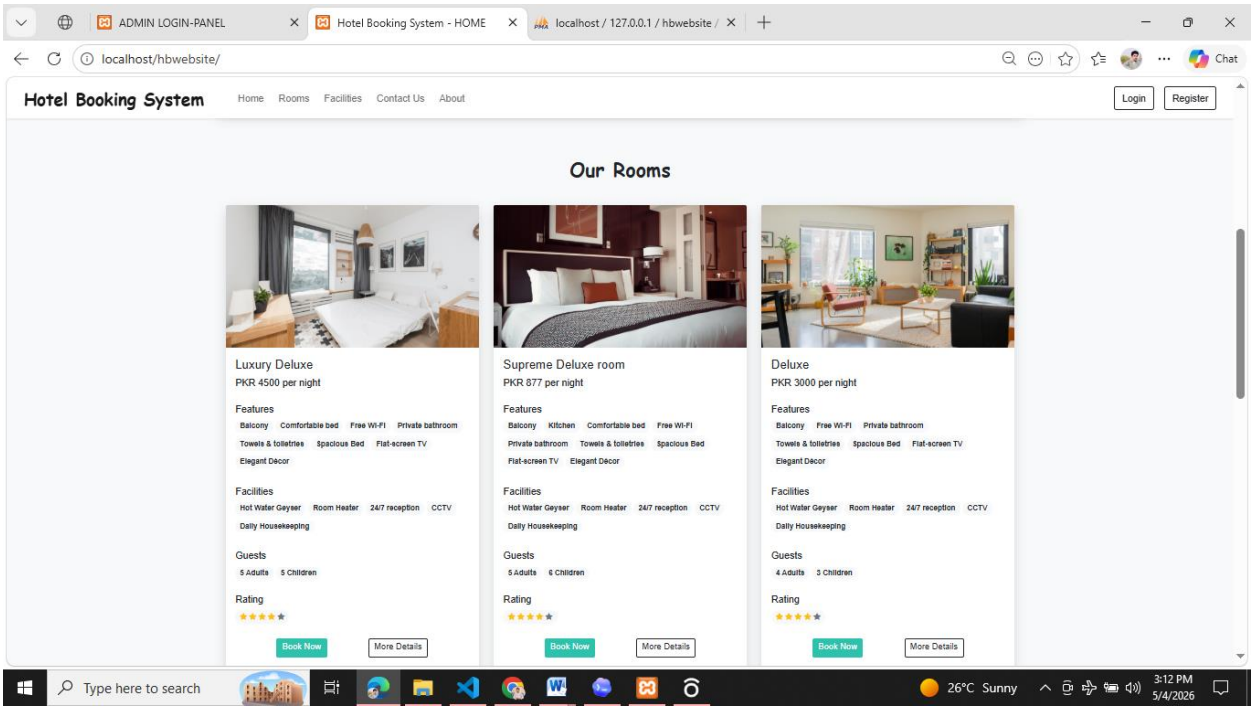
- **Home page**



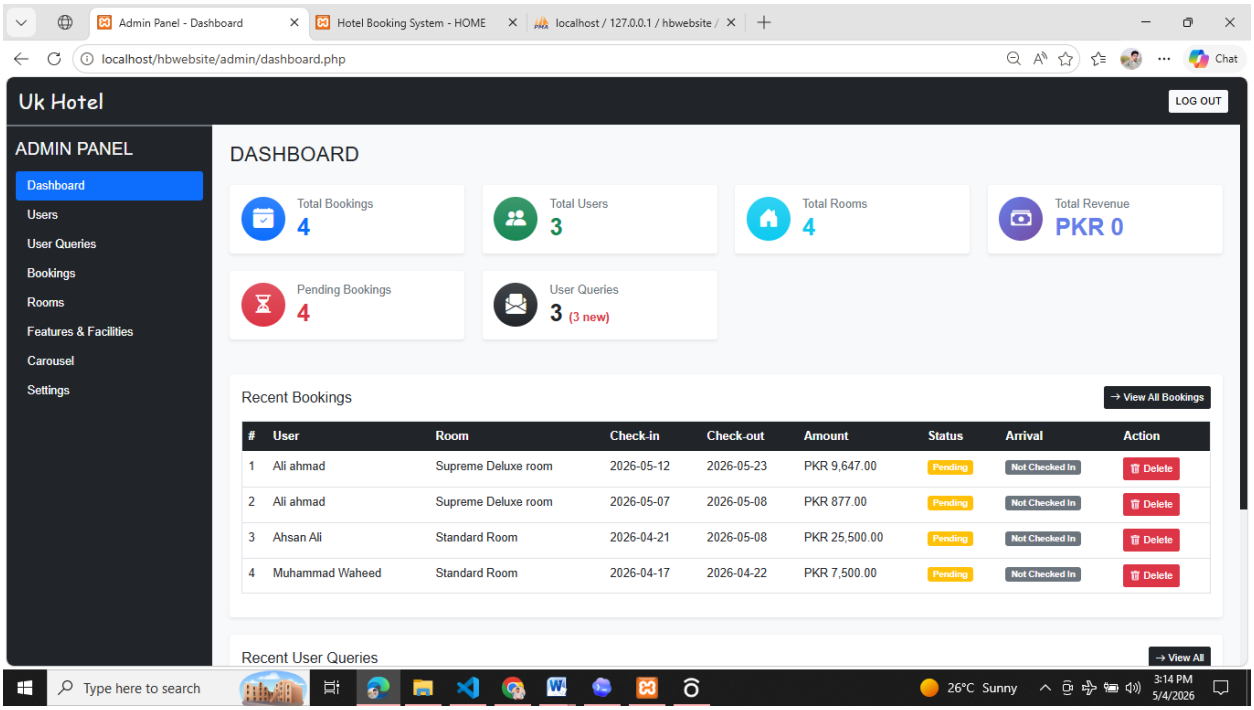
- **Login/Register**



- **Room listing**



- Room details
- Payment flow
- Invoice
- Booking history
- **Admin dashboard**



4.5 Screen Objects and Actions

The following table details the primary UI elements and the specific user or system actions associated with each screen in the application.

Screen	Object	Action
Home	Availability Form	Filter rooms based on selected criteria.
Header	Login / Register	Open authentication modals for user access.

Screen	Object	Action
Rooms	Filters	Apply AJAX-based filtering to update room lists dynamically.
Room Details	Date Input	Automatically calculate total stay amount based on selection.
Room Details	Pay Now	Initiate the booking process and redirect to payment.
Bookings	Invoice	Generate and view the transaction invoice.
Admin Rooms	CRUD Buttons	Perform Create, Read, Update, and Delete operations to manage room inventory.

4.6 Test Cases

- **TC-01:** Registration success → account created
- **TC-02:** Duplicate email → error
- **TC-03:** Login success → session created
- **TC-04:** Wrong password → error
- **TC-05:** Room filter → booked rooms excluded
- **TC-06:** Invalid dates → blocked
- **TC-07:** Payment success → booking paid
- **TC-08:** Payment cancel → pending
- **TC-09:** Admin add room → success
- **TC-10:** Toggle room status → updated